

Manual Penggunaan — Simulator UGV Off-Road (Traversability-Aware Speed Control)

Dokumen ini menjelaskan **cara menjalankan** simulator, bukan hasil eksperimennya. Ditujukan untuk orang yang baru pertama kali membuka folder `ugv_sim_2d/` — misalnya dosen pembimbing/penguji atau mahasiswa lain — agar bisa menjalankan simulasi dari nol tanpa membaca kode.

Catatan cakupan (baca dulu). Folder ini berisi **9 file modul** dan **dua entry point nyata**: dashboard interaktif (`dashboard.py`) dan pustaka simulasi (`simulate.py`). Tidak ada file `main.py`, `plot.py`, maupun `results_io.py` di folder ini; dengan demikian tidak ada mode batch yang otomatis menulis `hasil_simulasi.csv`, `config_simulasi.json`, atau 9 berkas PNG. Manual ini hanya mendokumentasikan yang benar-benar ada di kode. Bila Anda memerlukan ekspor CSV/PNG otomatis, lihat [Lampiran A](#).

Daftar Isi

1. Ringkasan Singkat
2. Prasyarat & Instalasi
3. Mode 1 — Dashboard Interaktif (`dashboard.py`)
4. Mode 2 — Simulasi Batch via `simulate.py`
5. Mengubah Parameter Simulasi (Tuning)
6. Interpretasi Output
7. Troubleshooting Singkat
8. Lampiran A — Menyimpan Hasil ke CSV

1. Ringkasan Singkat

Simulator ini memodelkan sebuah **UGV (Unmanned Ground Vehicle)** yang bernavigasi di medan off-road 2D. Peta direpresentasikan sebagai **grid 80 kolom × 60 baris** dengan `CELL_SIZE = 0.5 m`, sehingga area dunia = **40 m × 30 m**. Medan terdiri atas lima kelas (`grass`, `sand`, `puddle`, `rubble`, `mud`) plus `obstacle`.

Alur inti simulasi:

1. **Bangun peta terrain** — blob acak reproducible (`BLOB_SEED = 42`).
2. **Hitung skor traversabilitas T** tiap sel lewat formula 6-langkah (`terrain.py`).
3. **Rencanakan jalur referensi** dengan **A* 8-arah** berbiaya `1/T` (sel T rendah = mahal), lalu dihaluskan dan diresampling (`pathplanning.py`).
4. **Kendalikan kecepatan** UGV mengikuti jalur dengan salah satu dari tiga skenario kontrol: - **P-Controller** — $v = v_{\max} \cdot T$, heading lewat P-gain. - **MPC (no T)** — MPC tanpa batasan traversabilitas. - **MPC + T** — MPC dengan constraint $v(k) \leq v_{\max} \cdot T(k)$.

Tujuan simulator: membandingkan ketiga skenario dari sisi profil kecepatan, kestabilan Lyapunov, pelanggaran constraint (violations), dan penelusuran jalur.

2. Prasyarat & Instalasi

2.1 Python

Gunakan **Python 3.10 atau lebih baru** (disarankan 3.11+). Batas versi minimum tidak ditulis eksplisit dalam kode, tetapi `requirements.txt` menetapkan `numpy>=1.26`, `pandas>=2.0`, dan `scipy>=1.11` yang secara praktis membutuhkan Python 3.10+.

Periksa versi Python Anda:

```
python --version
```

2.2 Dependency

Folder ini menyertakan `requirements.txt` :

Paket	Versi minimum
streamlit	<code>>=1.32</code>
numpy	<code>>=1.26</code>
pandas	<code>>=2.0</code>
matplotlib	<code>>=3.8</code>
scipy	<code>>=1.11</code>

Cara paling aman — buat virtual environment lalu install dari `requirements.txt` :

```
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

Jika `requirements.txt` tidak tersedia, install manual:

```
pip install "streamlit>=1.32" "numpy>=1.26" "pandas>=2.0" "matplotlib>=3.8" "scipy>=1.11"
```

2.3 Masuk ke direktori yang benar

Seluruh perintah harus dijalankan dari **DALAM folder** `ugv_sim_2d/` . Modul saling mengimpor secara langsung (`import config`, `import terrain`, `dst.`), sehingga menjalankan dari direktori lain akan menyebabkan error impor.

```
cd path/ke/traversability_pipeline/ugv_sim_2d
```

3. Mode 1 — Dashboard Interaktif (`dashboard.py`)

Ini adalah cara termudah dan paling lengkap secara visual. Dari dalam folder `ugv_sim_2d/` :

```
streamlit run dashboard.py
```

Streamlit akan mencetak URL lokal (default `http://localhost:8501`). Buka URL tersebut di browser.

3.1 Parameter di sidebar

Semua kontrol berada di sidebar kiri, di bawah judul **"Parameter Simulasi"**:

Kontrol	Tipe	Rentang / Pilihan	Default
<code>v_max</code> (m/s)	slider	0.5 – 3.0, langkah 0.1	2.0
Drivetrain	radio	2WD / 4WD	4WD
Skala Terrain (%)	slider	50 – 150, langkah 5	100
Skenario Kontrol	selectbox	P-Controller / MPC (no T) / MPC + T	P-Controller

- `v_max` menetapkan kecepatan maksimum kendaraan; `v_max · T` menjadi batas kecepatan per medan.
- **Drivetrain** memengaruhi `capability` (2WD = 0.50, 4WD = 0.90) sehingga mengubah skor T tiap medan.
- **Skala Terrain** memperbesar/memperkecil radius blob medan (150% = medan lebih besar dan lebih sulit).
- **Skenario Kontrol** memilih skenario mana yang ditonjolkan pada plot trajektori/kecepatan; ketiga skenario tetap dihitung (lihat 3.2).

3.2 Tombol "Jalankan"

Menekan **"▶ Jalankan"** menjalankan **ketiga skenario sekaligus** (P, MPC, MPC+T) pada parameter yang sama, agar bisa dibandingkan berdampingan. Setelah selesai, dashboard menampilkan kartu metrik, tabel & bar chart kecepatan per terrain untuk ketiga skenario, plot profil kecepatan `v(t)` , plot Lyapunov `V(e)` , serta **tayangan ulang lintasan** (slider Frame) untuk skenario yang dipilih di sidebar.

3.3 Panel "Kecepatan Batas per Terrain"

Di bagian bawah sidebar terdapat panel **"Kecepatan Batas per Terrain"**. Panel ini menampilkan `v_max · T` untuk setiap jenis medan (grass, sand, puddle, rubble, mud) secara **real-time** — nilainya langsung berubah begitu Anda menggeser `v_max` atau mengganti drivetrain, tanpa perlu menekan "Jalankan".

3.4 Peta yang selalu tampil

Peta terrain, heatmap traversabilitas, panjang lintasan, dan jumlah waypoint dimuat dan ditampilkan otomatis (di-cache) begitu halaman dibuka, bahkan sebelum "Jalankan" ditekan.

3.5 Menghentikan dashboard

Kembali ke terminal tempat Streamlit berjalan, tekan `Ctrl + C`.

4. Mode 2 — Simulasi Batch via `simulate.py`

`simulate.py` adalah **pustaka fungsi** (bukan skrip dengan blok `if __name__ == "__main__"`). Untuk menjalankan simulasi non-interaktif dari terminal, panggil fungsi-fungsinya dari sebuah skrip Python singkat atau dari sesi interaktif Python. Semua log kemajuan (progress) tercetak ke terminal oleh fungsi-fungsi tersebut.

4.1 Fungsi publik yang tersedia di `simulate.py`

Fungsi	Kegunaan
<code>run_simulation(scenario, drivetrain, terrain_grid, T_grid, path_xy, path_s, path_x, path_y, path_th, mpc_params=None)</code>	Menjalankan satu run; mengembalikan objek <code>RunResult</code> . Mencetak ringkasan (jumlah step, panjang tercapai, violations, status goal).
<code>autotune_mpc(...)</code>	Grid search kecil parameter MPC (<code>Np</code> , <code>Q_xy</code> , <code>Rv</code>) sampai konvergen. Mengembalikan dict parameter terbaik.
<code>compute_metrics(run_result)</code>	Menghitung metrik lengkap dari sebuah <code>RunResult</code> ; mengembalikan dict.
<code>print_metrics_table(all_metrics)</code>	Mencetak tabel metrik banyak run ke terminal.
<code>verify_all_reached_goal(results)</code>	Verifikasi semua run mencapai goal; cetak status per run; kembalikan <code>bool</code> .
<code>verify_terrain_coverage(results)</code>	Verifikasi tiap run melewati kelima kelas terrain.
<code>diagnose_mpc_failure(results)</code>	Cetak diagnosis untuk run yang gagal mencapai goal.

Skenario valid: `"P"`, `"MPC"`, `"MPCT"`. Drivetrain valid: `"2WD"`, `"4WD"`.

4.2 Contoh skrip batch (jalankan 6 run: 3 skenario × 2 drivetrain)

Buat file `jalankan_batch.py` **di dalam folder** `ugv_sim_2d/`, isi dengan:

```

import config
import terrain as trn
import pathplanning as pp
import simulate as sim

# 1) Bangun peta terrain + tabel T + grid T (contoh: skala 100%)
terrain_grid = trn.build_terrain_grid(terrain_scale=1.0)
T_table = trn.build_T_table()
trn.print_T_table(T_table) # cetak tabel skor T ke terminal (opsional)

results = []
all_metrics = {}

for drivetrain in ["2WD", "4WD"]:
    # Grid T bergantung drivetrain (capability berbeda)
    T_grid = trn.build_T_grid(terrain_grid, drivetrain, T_table)

    # 2) A* + smoothing + resampling → jalur referensi
    path_xy, path_s, path_x, path_y, path_th = pp.full_pipeline(
        T_grid,
        (config.START_ROW, config.START_COL),
        (config.GOAL_ROW, config.GOAL_COL))

    # 3) Jalankan tiga skenario
    for scenario in ["P", "MPC", "MPCT"]:
        r = sim.run_simulation(
            scenario, drivetrain, terrain_grid, T_grid,
            path_xy, path_s, path_x, path_y, path_th)
        results.append(r)
        all_metrics[r.label] = sim.compute_metrics(r)

# 4) Verifikasi & tabel metrik
sim.verify_all_reached_goal(results)
sim.verify_terrain_coverage(results)
sim.print_metrics_table(all_metrics)
sim.diagnose_mpc_failure(results) # hanya mencetak jika ada yang gagal

```

Jalankan dari dalam folder `ugv_sim_2d/` :

```
python jalankan_batch.py
```

4.3 Urutan proses yang tercetak di terminal

Berdasarkan log yang dihasilkan fungsi-fungsi di atas, Anda akan melihat kira-kira urutan berikut:

1. (Opsional) **Tabel skor T** per terrain untuk 2WD & 4WD — dari `print_T_table`.
2. Untuk tiap run: baris `[<label>] ...` diikuti ringkasan `N steps | s=.../...m (..%) | viols=.. | goal=OK/FAIL` — dari `run_simulation`.
3. **Status goal per run** — dari `verify_all_reached_goal`.
4. **Coverage terrain per run** — dari `verify_terrain_coverage`.
5. **Tabel metrik lengkap** semua run — dari `print_metrics_table`.
6. **Diagnosis** (hanya jika ada run yang gagal mencapai goal) — dari `diagnose_mpc_failure`.

4.4 Auto-tuning MPC (opsional)

Jika sebuah skenario MPC sulit konvergen dengan parameter default di `config.py`, panggil `autotune_mpc(...)` untuk mencari parameter yang lolos kriteria (goal tercapai, RMSE sebanding P-Controller, Lyapunov cukup monoton), lalu teruskan hasilnya sebagai `mpc_params` ke `run_simulation`. Grid pencarian diatur oleh `TUNE_Np`, `TUNE_Q_xy`, dan `TUNE_Rv` di `config.py`.

4.5 Lama proses

Kode tidak memuat estimasi waktu eksplisit. Sebagai gambaran wajar: run **P-Controller** paling cepat; run **MPC** dan **MPC + T** lebih lambat karena setiap langkah memanggil solver `SLSQP` (`scipy.optimize.minimize`, `maxiter=50`) sepanjang horizon. Batas atas jumlah langkah per run adalah `MAX_STEPS = 3500`. Menjalankan enam run berurutan sangat bergantung pada mesin Anda; siapkan waktu beberapa puluh detik hingga beberapa menit.

5. Mengubah Parameter Simulasi (Tuning)

`config.py` adalah satu-satunya file yang perlu diedit untuk tuning/eksperimen. Setelah mengedit `config.py` :

- untuk **Mode 2 (batch)** → jalankan ulang skrip Python Anda;
- untuk **Mode 1 (dashboard)** → **restart** proses Streamlit (`Ctrl + C` lalu jalankan kembali `streamlit run dashboard.py`) agar nilai baru terbaca.

Parameter yang paling umum diubah:

Parameter	Arti	Default
START_WORLD / GOAL_WORLD	Koordinat dunia (x, y) titik awal & tujuan (m)	(2.0, 15.0) / (38.0, 15.0)
NROWS / NCOLS	Dimensi grid (baris × kolom)	60 / 80
CELL_SIZE	Meter per sel	0.5
CAPABILITY	Kapabilitas traksi 2WD / 4WD	{"2WD": 0.50, "4WD": 0.90}
BLOB_SEED	Seed RNG peta (reproduktibilitas)	42
BLOB_SPECS	Spesifikasi blob medan (label, jumlah, r_min, r_max)	lihat config.py
N_OBSTACLES	Jumlah kotak obstacle	8
V_MAX	Kecepatan maksimum (m/s)	2.0
MPC_Np / MPC_Nc	Horizon prediksi / kontrol MPC	15 / 6
MPC_Q_xy / MPC_Q_th	Penalti error posisi / heading	20.0 / 2.0
MPC_Rv / MPC_Rw	Penalti effort v / omega	0.05 / 0.10
T_RISK	Ambang T untuk risk event	0.30
MAX_STEPS	Batas langkah per run	3500

Catatan drivetrain di dashboard. Nilai `V_MAX` di dashboard di-override sementara oleh slider `v_max` ; jadi untuk eksperimen dashboard cukup gunakan slider, sedangkan untuk batch ubah `V_MAX` di `config.py` .

6. Interpretasi Output

6.1 Metrik dari `compute_metrics(...)`

Fungsi `compute_metrics` mengembalikan dict berisi metrik berikut (kunci = nama kolom yang tercetak oleh `print_metrics_table`):

Kolom	Arti
Waktu (s)	Durasi run (waktu simulasi terakhir)
Jarak (m)	Total panjang lintasan aktual yang ditempuh
v_avg (m/s)	Kecepatan rata-rata
RMSE path (m)	RMSE jarak ke jalur referensi
MAE path (m)	MAE jarak ke jalur referensi
$ a _{avg}$ (m/s ²)	Percepatan absolut rata-rata
Risk events / Risk (%)	Jumlah / persentase langkah dengan $T < T_RISK$
Violations / Violation (%)	Jumlah / persentase langkah dengan $v > v_max \cdot T + VIOL_TOL$
T_avg / T_min	Rata-rata / minimum skor T sepanjang lintasan
V_lyap_0 / V_lyap_f	Nilai Lyapunov awal / akhir
Lyap mono (%)	Persentase langkah dengan $\Delta V \leq 0$ (kestabilan)
Path frac (%)	Fraksi panjang jalur yang dicapai
Goal reached	YES / NO
Terrain covered	Daftar terrain yang dilewati
v_grass ... v_mud	Kecepatan rata-rata per jenis terrain

6.2 Objek `RunResult`

`run_simulation` mengembalikan `RunResult` yang menyimpan array time-series penuh (`t`, `px`, `py`, `theta`, `v`, `omega`,

`T_arr`, `V_lyap`, `pos_err`, `v_ratio`) plus `viols`, `risk_events`, `reached_goal`, dan `path_frac`. Objek inilah yang menjadi sumber seluruh plot dashboard dan seluruh metrik.

6.3 Penyimpanan hasil

Penting. Kode yang ada **tidak menulis file hasil** (tidak ada `hasil_simulasi.csv`, `config_simulasi.json`, `metrics.csv`, maupun PNG). Semua metrik hanya **dicetak ke terminal** (batch) atau **ditampilkan di layar** (dashboard). Jika Anda ingin menyimpan hasil ke berkas, lakukan ekspor manual — lihat [Lampiran A](#). Karena tidak ada penulisan berkas otomatis, tidak ada risiko file hasil tertimpa; namun juga tidak ada riwayat yang tersimpan kecuali Anda mengeksportnya sendiri.

7. Troubleshooting Singkat

- **ModuleNotFoundError** → dependency belum terpasang. Aktifkan virtual environment lalu `pip install -r requirements.txt`.
- **Error impor / ImportError / FileNotFoundError saat start** → hampir selalu karena dijalankan dari direktori yang salah. Pastikan Anda berada **di dalam** folder `ugv_sim_2d/` sebelum menjalankan perintah apa pun.
- **RuntimeError: A* tidak menemukan jalur ...** → obstacle/parameter medan memblokir seluruh jalur (misalnya `Skala Terrain` terlalu besar, atau `START_WORLD / GOAL_WORLD` diletakkan di dalam obstacle). Kecilkan skala terrain atau pindahkan start/goal.
- **Run gagal mencapai goal setelah edit parameter ekstrem** → `run_simulation` mencetak `goal=FAIL` dan `verify_all_reached_goal` menandai run tersebut. Jalankan `diagnose_mpc_failure(results)` untuk melihat kemungkinan penyebab (mis. kecepatan terlalu rendah → naikan `Q_xy` /turunkan `Rv`; Lyapunov tidak monoton → periksa lookahead). Untuk MPC, coba `autotune_mpc(...)`.
- **Dashboard tidak memperbarui nilai config.py** → restart Streamlit (Ctrl + C lalu jalankan ulang).

Lampiran A — Menyimpan Hasil ke CSV

Karena kode tidak menulis berkas hasil secara otomatis, gunakan `pandas` untuk mengeksport metrik dari skrip batch Anda (lanjutan dari contoh di [bagian 4.2](#)):

```
import pandas as pd

# all_metrics: dict {label -> dict metrik} dari contoh bagian 4.2
df = pd.DataFrame(all_metrics).T          # baris = label run, kolom = metrik
df.to_csv("hasil_metrik.csv", index_label="run")
print("Tersimpan: hasil_metrik.csv")
```

Untuk menyimpan time-series penuh sebuah run tertentu:

```
r = results[0] # contoh run pertama
pd.DataFrame({
    "t": r.t, "px": r.px, "py": r.py, "theta": r.theta,
    "v": r.v, "omega": r.omega, "T": r.T_arr,
    "V_lyap": r.V_lyap, "pos_err": r.pos_err, "v_ratio": r.v_ratio,
}).to_csv(f"timeseries_{r.label}.csv", index=False)
```

Anda bertanggung jawab memilih nama berkas; skrip di atas **akan menimpa** berkas dengan nama sama bila dijalankan ulang — backup manual bila perlu menyimpan beberapa eksperimen.